```
        .Where(x => x.RecipeId == id)                                   ❸
        .Select(x => new RecipeDetailViewModel                          ❹
        {                                                               ❹
            Id = x.RecipeId,                                            ❹
            Name = x.Name,                                              ❹
            Method = x.Method,                                          ❹
            Ingredients = x.Ingredients                                 ❺
            .Select(item => new RecipeDetailViewModel.Item              ❺
            {                                                           ❺
                Name = item.Name,                                       ❺
                Quantity = $"{item.Quantity} {item.Unit}"               ❺
            })                                                          ❺
        })
        .SingleOrDefaultAsync();                                        ❻
}
```

❶ The id of the recipe to load is passed as a parameter.

❷ As before, a query starts from a DbSet property.

❸ Limits the query to the recipe with the provided id

❹ Maps the Recipe to a RecipeDetailViewModel

❺ Loads and maps linked Ingredients as part of the same query

❻ Executes the query and maps the data to the view model

Notice that as well as mapping the `Recipe` to a `RecipeDetailViewModel`, you map the related `Ingredient`s for a `Recipe`, as though you're working with the objects directly in memory. One advantage of using an ORM is that you can easily map child objects and let EF Core decide how best to build the underlying queries to fetch the data.

NOTE EF Core logs all the SQL statements it runs as `LogLevel.Information` events by default, so you can easily see what queries are running against the database.

Your app is definitely shaping up. You can create new recipes, view them all in a list, and drill down to view individual recipes with their ingredients and